



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3
«Оценка ошибки округления и ошибки в передаваемых
данных»
по курсу «Численные методы линейной алгебры»

Студент группы ИУ9-71Б Лысенко В. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Цель работы

Реализовать автоматическое вычисление оценки относительной ошибки округления и ошибки в передаваемых данных.

2 Задачи

1. Реализовать автоматическое вычисление оценки относительной ошибки округления и ошибки в передаваемых данных для больших объёмов данных.
2. Реализовать автоматическое вычисление оценки относительной ошибки округления и ошибки в передаваемых данных для значений из индивидуального варианта.
3. Сравнение вычисленных данных с полученными на бумаге.

3 Практическая реализация

Для получения ошибки округления необходима функция, с погрешностью вычисляющая вектор x в выражении $Ax = f$. В качестве таковой была взята функция из предыдущей лабораторной работы, реализующая классический метод Гаусса. Данный метод был переопределён таким образом, чтобы возвращать помимо вектора x ещё и $g(A)$ (коэффициент роста).

При вычислении ошибки передаваемых данных для получения вектора x использовались уже имеющиеся возможности языка.

При выдаче задания было сказано находить норму по формуле Евклида, однако в письменной работе использовалась формула равномерной нормы, поэтому в данной лабораторной работе были реализованы методы для нахождения обеих норм (`norm()` и `matrix-norm()` - для поиска норм вектора и матрицы по Евклиду, `inf-norm()` и `inf-matrix-norm()` - для поиска равномерных норм вектора и матрицы).

Ниже представлен код программы:

```
n = 2
p = 2.0
t = 26
cup = 100

function gauss_classic(A, F)
    max = maximum(abs.(A))
    max_elems = zeros(Float64, n-1)

    for k in 1:n-1
        for i in k+1:n
            factor = A[i, k] / A[k, k]
            A[i, k:end] -= factor * A[k, k:end]
            F[i] -= factor * F[k]
        end

        max_elems[k] = maximum(abs.(A)) / max
    end

    g_a = maximum(max_elems)

    X = zeros(Float64, n)
    for i in n:-1:1
        X[i] = F[i] / A[i, i]
    end
end
```

```

        for j in i-1:-1:1
            F[j] -= A[j, i] * X[i]
        end
    end

    return g_a, X
end

function norm(x)
    sum = 0.0
    for element in x
        sum += element^2
    end
    return sqrt(sum)
end

function matrix_norm(A)
    sum = 0.0
    for i in 1:n
        for j in 1:n
            #print(A[i, j], " ")
            sum += A[i, j]^2
        end
        #println()
    end
    return sqrt(sum)
end

function inf_norm(x)
    return maximum(abs.(x))
end

function inf_matrix_norm(A)
    return maximum([sum(abs.(A[i, :])) for i in 1:size(A, 1)])
end

#X = ones(Float64, n)
#A = rand(-cup:cup, n, n)
#A = Float64.(A)
#F = [Float64(x) for x in (A * X)]

X = [2.625 1.5]
A = [[48.0, 340.0] [340.0, 23.0]]

```

```

F = [636.0 927.0]

println("-----")

g_A, X0 = gauss_classic(copy(A), copy(F))

delta_X = vec(X) - X0

norm_X = inf_norm(X)
norm_delta_X = inf_norm(delta_X)

error = norm_delta_X / norm_X

nu_A = inf_matrix_norm(A) * inf_matrix_norm(inv(A))

#println("X = ", X)
#println("X0 = ", X0)
#println("delta_X = ", delta_X)
println("norm(delta_X) = ", norm_delta_X)

println()

println("nu(A) = ", nu_A)
println("n = ", n)
println("g(A) = ", g_A)
println("p^(-t) = ", p^(-t))

rounding_error = nu_A * n * g_A * (p^(Float64(-t)))

println("\n", error, " <= ", rounding_error, "\t<--- rounding_error")
println("-----")

A1 = copy(A)
F1 = copy(F)
X1 = copy(X)

delta_A1 = zeros(Float64, n, n)
delta_F1 = zeros(Float64, n)
#delta_X1 = zeros(Float64, n)

delta_A1 .+= 25
delta_F1 .+= 2.6

wrong_A1 = A1 + delta_A1

```

```
wrong_F1 = vec(F1) + vec(delta_F1)

X01 = wrong_A1 \ wrong_F1
#println(X01)

delta_X1 = vec(X1) - vec(X01)
#println(delta_X1)
nu_A1 = inf_matrix_norm(A1) * inf_matrix_norm(inv(A1))

error1 = inf_norm(delta_X1) / inf_norm(X1)

data_error = nu_A1 * ( inf_norm(delta_F1) / inf_norm(F1) + inf_matrix_norm(
    delta_A1) / inf_matrix_norm(A1))
println("\n", error1, " <= ", data_error, "\t<--- data_error")
```

4 Результаты

Результат с данными из индивидуального варианта:

```
nu(A) = 1.3148406931246508
n = 2
g(A) = 7.015686274509803
p^(-t) = 1.4901161193847656e-8

5.075305255429287e-16 <= 2.7491181504492666e-7 <--- rounding_error
-----

0.0935823072300589 <= 0.1731260284743817 <--- data_error
```

Как видно, результаты совпали с вычисленными на бумаге.

Результат с большими данными (размерность матрицы и векторов 100):

```
norm(delta_X) = 7.177591854201637e-13

nu(A) = 2415.918409134761
n = 100
g(A) = 5457.88234993616
p^(-t) = 1.4901161193847656e-8

7.177591854201637e-13 <= 19.648370808515637 <--- rounding_error
-----

38.06860185724293 <= 1044.734841390132 <--- data_error
```

5 Выводы

В ходе выполнения лабораторной работы было реализовано автоматическое вычисление оценки относительной ошибки округления и ошибки в передаваемых данных для больших объёмов данных. Корректность работы программы была так же протестирована на данных из индивидуального варианта, в результате чего полученный результат сошёлся с ожидаемым.